IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

LLM-Flax: Reducing Domain Engineering in Neuro-Symbolic Task Planning with Large Language Models

SEONGMIN KIM[1] and DAEGYU LEE[2]
[1]Department of Physical AI Engineering, Jeonbuk National University (JBNU), Jeonju, Republic of Korea
[2]Department of Advanced Defense Technology and Industry, Jeonbuk National University (JBNU), Jeonju, Republic of Korea

Corresponding author: Daegyu Lee (e-mail: lee.dk@jbnu.ac.kr).

ABSTRACT Adapting a neuro-symbolic task planner to a new domain can require hand-authored symbolic rules and solved training instances for a learned object scorer. This paper presents LLM-Flax, which applies a locally hosted large language model (LLM) to three parts of the Flax planning pipeline: offline generation of relaxation and complementary rules, budget-gated recovery after a planning timeout, and zero-shot object scoring in place of a trained Graph Neural Network (GNN). Rule outputs are constrained by a JSON schema and checked through validation and corrective prompting before use. Across eight MazeNamo benchmarks, the Stage 1 configuration, which retains the trained GNN and replaces only the manual rules, obtains a mean success rate (SR) of 0.941, compared with 0.912 for manually authored rules. A controlled rule-set ablation changes SR by $+0.033$ (one additional solution out of 30) and incurs a 32% increase in planning time relative to the manual configuration. In a cross-domain probe using an identical untrained scorer for both rule sources, the generated rules match or exceed the manual rules on SokomindPlus and DifficultLogistics; the latter result is measured after correcting a planner-loop error that affected both configurations. Stage 2 preserves SR on the two evaluated benchmarks but provides no improvement, whereas Stage 3 obtains SR 0.533 on 12×12 Hard and SR 0.000 on 15×15 Hard without GNN training data. Diagnostic logs attribute the Stage 3 degradation to incomplete single-turn score generation, which leaves 70–99% of objects at the default score. Within the evaluated pipeline, schema-constrained rule generation is the only LLM use that matches the corresponding Flax component; runtime recovery and zero-shot object scoring remain preliminary.

INDEX TERMS Task Planning, Large Language Models, Neuro-Symbolic AI, PDDL, Automated Rule Generation

I. INTRODUCTION

TASK planning in complex, object-rich environments requires long-horizon reasoning over large state spaces, making classical symbolic planners computationally expensive [1], [2]. Neuro-symbolic methods reduce this search burden by using learned relevance estimates to select objects before invoking a symbolic solver [3], [4].

Flax [5] follows this design by combining a Graph Neural Network (GNN) object scorer with two forms of domain-specific symbolic knowledge: relaxation rules, which remove selected objects to construct a simpler problem, and complementary rules, which restore objects connected by spatial or structural relations. The original study reports that these components reduce planning time on object-rich maze tasks.

Adapting Flax to another domain nevertheless introduces two distinct costs. A domain expert must specify both rule types, and the GNN scorer requires a domain-specific set of solved planning instances. The first cost concerns symbolic knowledge engineering; the second concerns data generation and model training.

Large language models (LLMs) can produce structured outputs from formal specifications and may therefore reduce part of this adaptation cost [6], [7]. We examine three uses of an LLM within Flax, separating offline rule generation from two forms of runtime guidance.

The resulting framework, LLM-Flax, contains three stages:

- **Stage 1 — LLM Rule Generation:** The LLM maps a

PDDL domain definition to relaxation and complementary rules. Schema validation and corrective prompting reject malformed outputs before planning.

- **Stage 2 — LLM Failure Recovery:** After a timeout, the LLM selects excluded objects for one bounded replanning attempt. A feasibility check reserves time for the subsequent relaxation step.
- **Stage 3 — LLM Zero-Shot Object Scoring:** The LLM assigns object-relevance scores from the current problem state and goal, replacing the domain-trained GNN for this exploratory configuration.

Stage 1 addresses rule authoring while retaining the trained GNN. The combined Full LLM-Flax configuration removes both manual rules and GNN training data, but still requires the PDDL domain and problem descriptions used by any symbolic planner. Our experiments evaluate the stages separately because they have different costs and levels of empirical support.

II. RELATED WORK

A. NEURO-SYMBOLIC TASK PLANNING

Classical symbolic planners such as Fast Downward [1] and FF [8] search over formal state and action models, but grounding and search can become expensive as problem size grows. STRIPS [9] established the foundational formalism, while PDDLStream [10] extended it to continuous domains via blackbox samplers. Neuro-symbolic approaches address scalability by coupling neural components with symbolic solvers. PLOI [3] introduced GNN-based object filtering, reducing the grounded state space while preserving validity through iterative threshold relaxation. Subsequent work explored neuro-symbolic skills [11], relational transition models [12], predicate invention [13], and efficient abstract planning models [14]. LogiCity [15] further advances neuro-symbolic AI through large-scale abstract urban simulation.

Our work directly extends Flax [5], which combines a GNN object scorer with hand-crafted relaxation and complementary rules for object-rich PDDL instances. LLM-Flax retains its symbolic planning loop and studies the rule-authoring and training-data costs as separate problems.

B. LLMS FOR PLANNING AND FORMAL REASONING

Prior work has used LLMs as planners, translators, and reasoning components. SayCan [16] grounds language-model outputs in robot affordances, while LLM+P [17] translates natural-language tasks into PDDL problems for an external solver. PDDLego [18] and Guan et al. [19] use LLMs to construct or repair PDDL representations, whereas Tree-of-Thoughts [20] and chain-of-thought prompting [7] use language models to organize the reasoning process itself.

Stage 1 of LLM-Flax instead produces a compact rule configuration once per domain and leaves plan generation to Fast Downward within the Flax loop. Stages 2 and 3 query the LLM at runtime, but their outputs are restricted to object names or scalar relevance scores rather than actions or plans. The Stage 1 validator limits structural errors before a rule file is reused across task instances; semantic errors remain possible and are evaluated separately in Section V-C.

C. AUTOMATED KNOWLEDGE ENGINEERING

Automated PDDL domain acquisition [21] and rule learning [22] are established problems in AI planning. In contrast to learning a complete domain model from execution traces, Stage 1 maps an existing PDDL domain to the narrower JSON rule schema required by Flax [5].

III. BACKGROUND

A. PDDL PLANNING

A PDDL planning problem $\Pi = \langle \mathcal{D}, \tau \rangle$ consists of a domain $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ defining lifted predicates and actions, and a task instance $\tau = \langle \mathcal{O}, I, G \rangle$ specifying objects, initial state, and goal. A plan is a sequence of grounded actions mapping I to a state satisfying G.

B. NEURO-SYMBOLIC PLANNING WITH RULES

The baseline planner we extend, Flax [5], operates in three steps during inference:

Step 1 (GNN Pruning) A GNN assigns importance scores $s(o) \in [0, 1]$ to each object. Starting from threshold $q = 0.81$ with decay $\gamma = 0.9$, the planner iteratively lowers q and attempts to plan on the pruned object set $\mathcal{O}_1 = \{o \mid s(o) \geq q\}$ until success or timeout.

Step 2 (Relaxation) If Step 1 times out, relaxation rules remove certain objects from the full problem to create a simplified "rough" version. A rough plan μ_r is obtained quickly; objects in μ_r are merged back: $\mathcal{O}_2 = \mathcal{O}_1 \cup \{o \mid o \in \mu_r\}$.

Step 3 (Complementary Expansion) Complementary rules restore object pairs connected by structural constraints, yielding $\mathcal{O}_3 = \text{Comp}(\mathcal{O}_2)$, on which the final plan is computed.

Both rule types are stored as structured JSON configurations (see Section IV) and loaded at runtime. In the original system, these files are authored manually per domain.

IV. METHODOLOGY

A. RULE FORMAT

Relaxation rules Each rule specifies: (i) `pre_compute`—a binary predicate used to look up a related object (e.g., `oAt` maps an obstacle to its position); (ii) `precond`—unary predicate(s) identifying removable objects; (iii) `delete_objects`—argument index of the object to remove; (iv) `delete_effects`—literals to retract; (v) `add_effects`—literals to assert after removal. All values are integer argument indices (0-based).

The manually crafted relaxation rule for MazeNamo reads:

```
{"rule0": {"pre_compute": {"oat":[0,1]},
  "precond": {"islight":[0]},
  "delete_objects": [0],
  "delete_effects": {"islight":[0],
    "ismoveable":[0], "oat":[0,1]},
  "add_effects": {"posempty":[1]}}}
```

Semantically: for any light obstacle `o` at position `p`, remove `o` from the problem and mark `p` as empty.

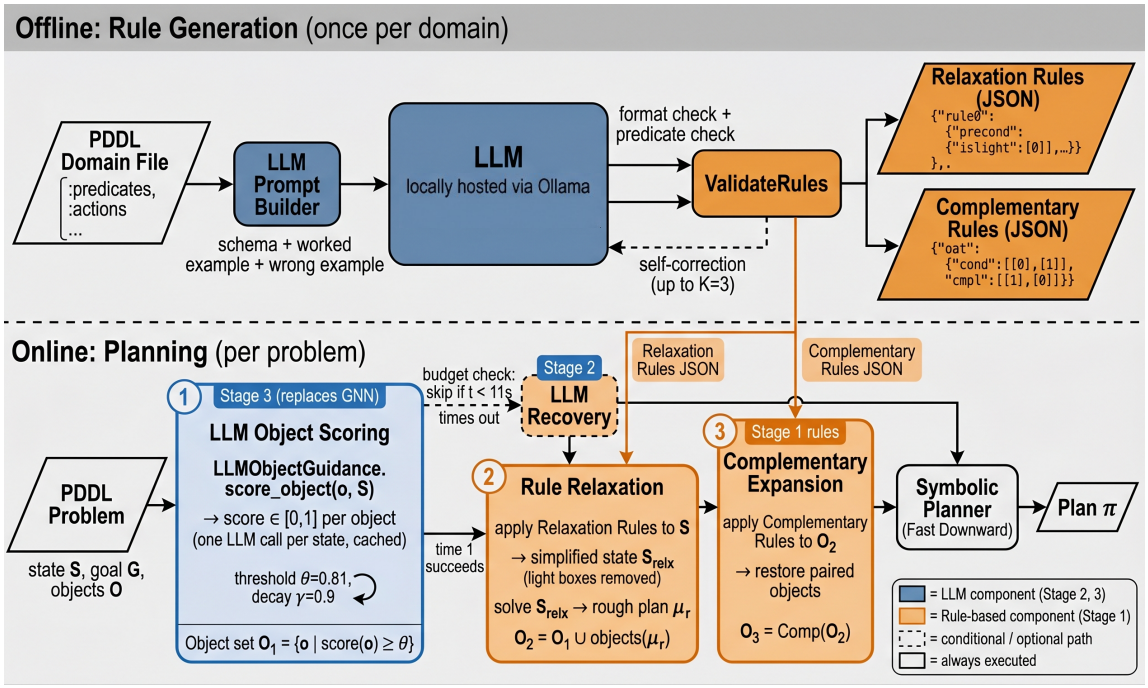

FIGURE 1: Architecture of LLM-Flax. *Top*: Stage 1 generates relaxation and complementary rules once per domain and validates the resulting JSON. *Bottom*: the Flax planning loop performs threshold-decay pruning, rule-based relaxation, and complementary expansion. Stage 2 optionally inserts a feasibility-gated recovery call after a pruning timeout; Stage 3 substitutes zero-shot LLM scores for the GNN scores. Stage 1 removes manual rule authoring, whereas Stage 3 removes GNN training.

Complementary rules Each entry maps a binary predicate name to paired `cond`/`cmpl` index lists. When an object at index `cond[i]` is included, the object at index `cmpl[i]` is also included. The manually crafted complementary rule is simply `{"oat": {"cond":[[0],[1]], "cmpl":[[1],[0]]}}`.

B. LLM RULE GENERATION

We use Gemma3-12B [23], hosted locally through Ollama's REST API, as the primary model. Generation uses temperature 0 and a 16,384-token context window. The model is queried twice per domain, once for each rule type.

Prompt design Each prompt defines the rule semantics and JSON schema, states that argument indices must be integers, and provides one valid example and one counterexample with string-valued indices. The complete PDDL domain definition is included so that predicate names and arities are available to the model.

Validation and self-correction LLM outputs are validated by ValidateRules, which checks: (i) all required keys are present; (ii) all index fields contain lists of integers; (iii) all predicate names exist in the domain (parsed from the PDDL `:predicates` block); and (iv) no duplicate rules appear. If validation fails, the error messages are fed back to the LLM as a correction turn, with up to $K = 3$ total attempts.

Post-processing Two deterministic safeguards are applied before the files are written:

- Deduplication: rules with identical JSON content (modulo key name) are merged.
- Predicate filtering: rules that reference predicates absent from the domain are removed.

The accepted rules are then saved as JSON files and loaded by the planner.

Algorithm 1 summarizes Stage 1.

C. STAGE 2: LLM-GUIDED FAILURE RECOVERY

Step 1 uses threshold decay to expand the selected object set after an unsuccessful planning attempt. This procedure does not distinguish among the excluded objects.

We introduce LLMRecoveryGuidance, inserted between a Step 1 timeout and the Step 2 relaxation fallback. Given the PDDL state, goal, included objects $\mathcal{O}_{cur}$, and excluded candidates $\mathcal{O}_{exc}$, the LLM selects excluded objects that may be needed for a plan. The LLM returns a JSON list of object names; the planner adds them and retries. If the retry succeeds the plan is returned; otherwise the pipeline falls through to Step 2 unchanged.

Budget policy LLM API calls impose a latency cost (≈3–5 s) that must be accounted for in per-problem timeout budgets. We adopt a three-part policy (see Section V-H for its budget analysis):

- Feasibility check: skip recovery if the remaining time before the Step 2 deadline is $< t_{\text{LLM}} + t_{\text{replan,min}} + t_{\text{step2,min}}$ (set conservatively to 11 s).

Algorithm 1: LLM-Flax Rule Generation (Stage 1)

Input : PDDL domain file $\mathcal{D}$; LLM $\mathcal{M}$; max retries K

Output: Relaxation rules R_{relx}; Complementary rules R_{cmpl}

$\mathcal{P}_{known} \leftarrow$ PARSEPREDICATES($\mathcal{D}$);
for *rule_type* $\in$ {*relaxation*, *complementary*} **do**
  prompt $\leftarrow$ BUILDPROMPT($\mathcal{D}$, rule_type);
  messages $\leftarrow$ [system, prompt];
  for $k = 1$ **to** K **do**
    raw $\leftarrow \mathcal{M}$(messages);
    $R \leftarrow$ PARSEJSON(raw);
    errors $\leftarrow$ VALIDATERULES(R, $\mathcal{P}_{known}$);
    if *errors* $= \emptyset$ **then break**;
    messages.append(raw, errors);
  $R \leftarrow$ DEDUP(R);
  $R \leftarrow$ FILTERUNKNOWN(R, $\mathcal{P}_{known}$);
return R_{relx}, R_{cmpl};

- Recovery cap: limit the recovery replan to 15% of total timeout.
- Step 2 guarantee: Step 2 always receives at least 5 s regardless of recovery outcome.

Under the evaluated timeout settings, this policy skips recovery for the 30 s case and permits a short recovery attempt for the 40 s case.

D. STAGE 3: LLM ZERO-SHOT OBJECT SCORING

The evaluated GNN scorer is trained on 200 solved problems from the target domain. Stage 3 removes this training requirement by replacing the GNN with LLMObjectGuidance, which implements the same `score_object(obj, state)` interface.

On the first call for a given state, the LLM receives the goal, the list of all objects with types, and up to 80 state facts, and returns a JSON dictionary mapping each object name to a score in $[0, 1]$. Scores are cached so only one LLM call is made per planning problem. The `train()` method performs no data collection or parameter updates. If the call fails, or if an object is absent from the returned dictionary, its score defaults to 0.5.

Full LLM-Flax Full LLM-Flax combines the three stages: Stage 1 generates rules from the domain definition, and Stages 2 and 3 provide runtime recovery and object scores. This configuration uses neither hand-written rules nor domain-specific GNN training data.

V. EXPERIMENTS

A. SETUP

Domain We evaluate on the MazeNamo benchmark [3]: a grid-based maze navigation task where a robot must reach a goal while manipulating heavy boxes (push-only) and light boxes (push or pick-and-place). We test on 10×10 (easy, medium, hard; n=50), 12×12 (medium, hard; n=50; expert n=30), and 15×15 (medium, hard; n=30) grids, using a GNN model trained on 200 problems. All reported runs use one random seed; SR differences are therefore presented as descriptive results rather than statistical significance claims.

Configurations Stage 1 is evaluated on all eight MazeNamo benchmarks. Stages 2 and 3 are evaluated on two selected benchmarks: Stage 2 where Step 1 timeouts occur frequently, and Stage 3 at two larger problem settings. Stage 1 compares two rule configurations, holding all other components fixed:

- **Manual**: Hand-crafted rules (1 relaxation, 1 complementary). Timeouts: 10 s / 30 s / 40 s for 10×10 / 12×12 / 15×15.
- **LLM-Flax**: LLM-generated rules (1 relaxation after dedup, 7 complementary). Same timeouts.

Stage 2 adds a third configuration, **LLM-Flax + Recovery**, which uses LLMRecoveryGuidance with the feasibility-gated budget policy described in Section IV-C. Stage 3 evaluates Full LLM-Flax—replacing the GNN with LLMObjectGuidance (zero-shot)—on two larger problem settings. Section V-G evaluates Stage 1 on DifficultLogistics and SokomindPlus with the same fixed, untrained scorer for both rule sources. This comparison is designed as a rule-transfer probe and is not directly comparable with the GNN-guided MazeNamo results.

Metrics

- **Success Rate (SR):** fraction of problems solved within the time budget.
- **Average Planning Time:** mean time over successful problems.
- **Average Plan Length:** mean number of actions in found plans.

Unless stated otherwise, planning time covers the complete planner call and therefore includes runtime LLM latency in Stages 2 and 3.

B. RESULTS

Table 1 reports the Stage 1 comparison, and Fig. 2 summarizes its SR values. Fig. 3 provides a separate qualitative trace of the zero-shot scoring interface.

The observed SR with LLM-generated rules is equal to or higher than that with manual rules on each benchmark. The two configurations tie on three tasks; on four others, the differences range from +0.020 to +0.040. The largest difference occurs on 15×15 Hard, where SR increases from 0.900 to 1.000. On 12×12 Expert, the corresponding values are 0.667 and 0.700; Section V-C examines this one-problem difference under controlled rule sets. The unweighted mean across the eight tasks is 0.941 for LLM-generated rules and 0.912 for manual rules. Fig. 4 provides a compact visual summary of the full result matrix.

C. RULE QUALITY ANALYSIS

Relaxation rules Gemma3-12B generates the same MazeNamo relaxation rule as the manual configuration (Table 2).

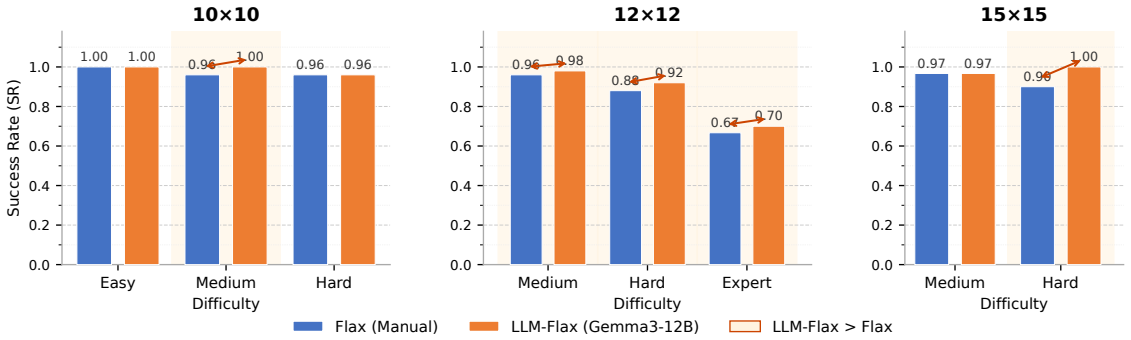

FIGURE 2: Success rate of manual and LLM-generated rules across the eight MazeNamo benchmarks. Both configurations use the same trained GNN scorer and planning budgets. Shading marks benchmarks on which the observed SR is higher with LLM-generated rules.

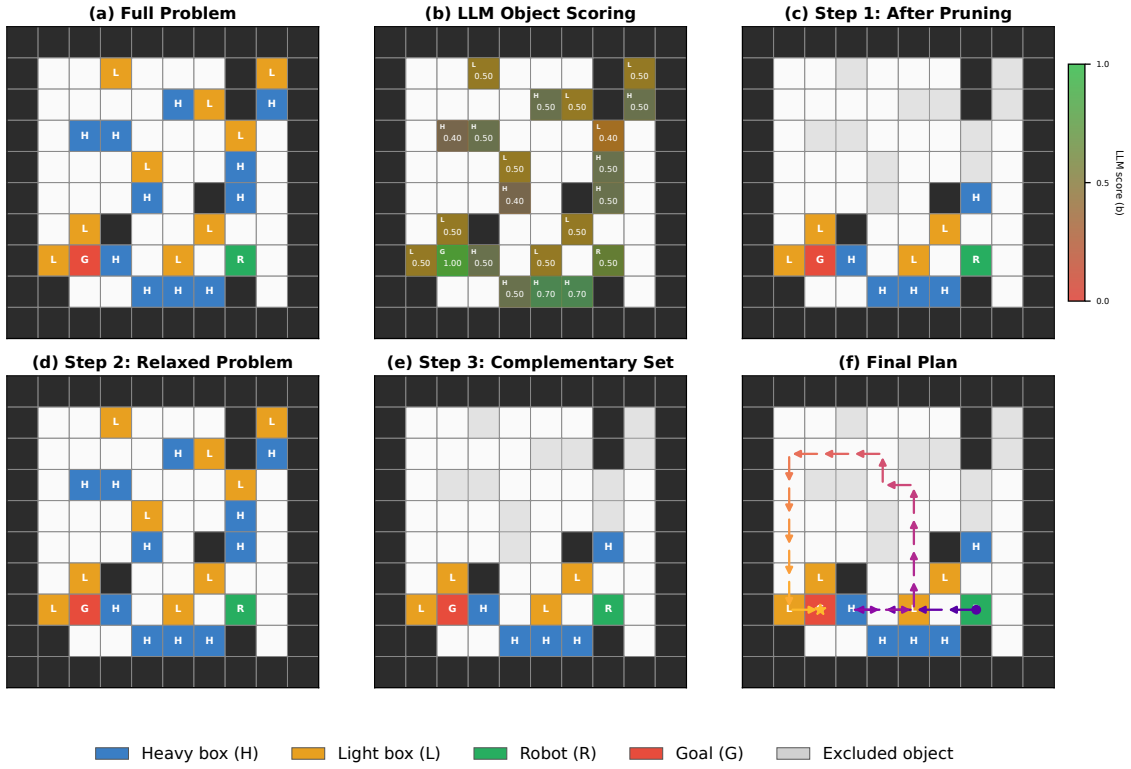

FIGURE 3: Illustrative trace using LLM-generated rules and precomputed zero-shot LLM scores on a 10×10 hard MazeNamo problem (problem 16, 163 objects). (a) Full problem with all objects. (b) Zero-shot relevance scores used for this trace. (c) After Step 1 threshold-based pruning: 21 objects excluded (grey), 142 retained at threshold 0.478. (d) Step 2 relaxed sub-problem: light boxes removed via relaxation rules, yielding a rough plan. (e) Step 3 complementary expansion: object pairs with structural constraints restored. (f) Final plan of length 39. The reported 0.8 s is planner time after score generation and is included for visualization only, not as an end-to-end Stage 3 result.

The `pre_compute`, `precond`, `delete_objects`, `delete_effects`, and `add_effects` fields are identical, so differences in the main comparison arise from the complementary rule sets or run-to-run planning behavior. Qwen2.5-14B adds a `clear:[0]` precondition that prevents removal of stacked light boxes. Llama3.1-8B and Mistral-7B produce additional rules with invalid indices, unknown predicates, or overly broad effects.

Complementary rules Gemma3-12B generates seven complementary rules (`upon`, `oat`, `rat`, `upto`, `downto`, `leftto`, `rightto`), whereas the manual configuration contains only `oat`. Because rule count and predicate coverage change together, the main comparison alone cannot attribute the SR difference to either factor. We therefore eval-

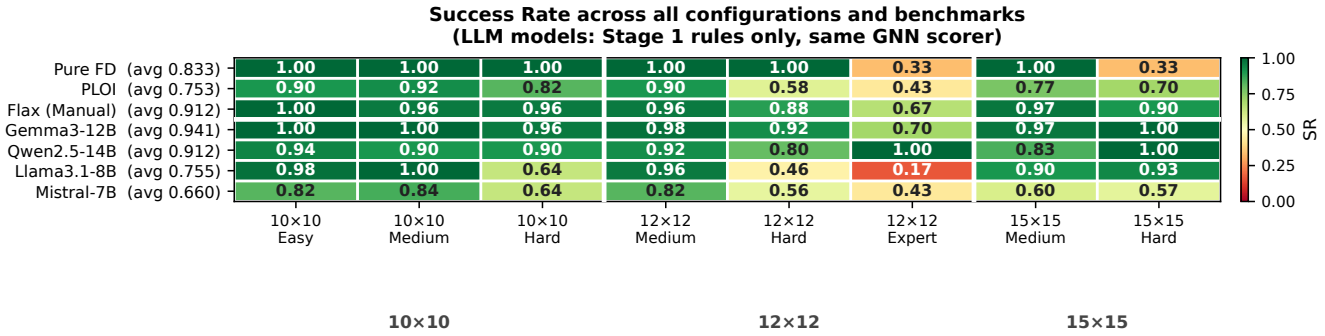

FIGURE 4: SR heatmap for Pure FD, PLOI, manual Flax, and four Stage 1 LLM rule generators across eight MazeNamo benchmarks. All LLM rows use the same trained GNN scorer; only the generated rules differ. The 12×12 Expert entries for Qwen2.5-14B, Llama3.1-8B, and Mistral-7B come from earlier runs and are not part of the controlled comparison in Section V-C.

TABLE 1: Flax (manual rules) vs. LLM-Flax (LLM-generated rules) within the Flax pipeline. SR = success rate (↑); Time = avg. planning time over successful problems in seconds (↓); Len = avg. plan length. Bold marks higher SR and lower time within each pair.

Task	Rules	SR	ΔSR	Time (s)	Len
10×10 Easy	Manual	**1.000**	±0.000	0.932	17.4
	LLM-Flax	**1.000**		**0.898**	17.1
10×10 Medium	Manual	0.960	+0.040	1.997	19.8
	LLM-Flax	**1.000**		**0.949**	16.6
10×10 Hard	Manual	**0.960**	±0.000	2.200	19.8
	LLM-Flax	**0.960**		**1.600**	19.3
12×12 Medium	Manual	0.960	+0.020	**1.914**	22.1
	LLM-Flax	**0.980**		2.151	22.3
12×12 Hard	Manual	0.880	+0.040	**3.965**	29.8
	LLM-Flax	**0.920**		4.529	29.2
12×12 Expert	Manual	0.667	+0.033	**4.528**	31.5
	LLM-Flax	**0.700**		5.982	29.3
15×15 Medium	Manual	**0.967**	±0.000	**8.740**	30.9
	LLM-Flax	**0.967**		9.007	31.0
15×15 Hard	Manual	0.900	+0.100	11.901	36.2
	LLM-Flax	**1.000**		**10.979**	36.0
Average (8 tasks)	Manual	0.912	+0.029	4.522	25.9
	LLM-Flax	**0.941**		**4.512**	25.1

uate four nested complementary-rule sets on 12×12 Expert (n=30), using the same relaxation rule, GNN checkpoint, and problem set (Table 3).

The count- and content-matched `oat`-only configuration reproduces the manual SR of 0.667. Adding `rat` and `upon` does not change SR. The full set, which adds four directional relations, solves one additional problem out of 30 (SR 0.700) and increases mean planning time by 32% relative to the manual configuration. This experiment bounds the observed benefit on this benchmark as small. Because the final condition adds four specific predicates at once, it does not separate aggregate rule count from the contribution of any individual directional relation.

TABLE 2: Relaxation rule preconditions generated by each model vs. the manual baseline. Gemma3-12B (the primary model) produces an exact match; others deviate.

Model	`precond`	Notes
Manual (baseline)	`{islight:[0]}`	Reference
Gemma3-12B	`{islight:[0]}`	Exact match
Qwen2.5-14B	`{islight:[0], clear:[0]}`	Over-conservative
Llama3.1-8B	`{islight:[0], ismoveable:[0]}`	+ 8 spurious rules
Mistral-7B	`{islight:[0]}`	Over-specified effects

TABLE 3: Rule-count ablation on 12×12 Expert (n=30, T=30 s), holding the relaxation rule fixed. `oat`-only matches Manual's complementary rule count and content.

Complementary rules	Count	SR	Time (s)
Manual (`oat`)	1	0.667	4.528
LLM, `oat`-only	1	0.667	4.403
LLM, `oat+rat+upon`	3	0.667	**4.300**
LLM, full (+4 direction rules)	7	**0.700**	5.982

D. STAGE 2: LLM-GUIDED FAILURE RECOVERY

Table 4 reports results on the two benchmarks where GNN Step 1 most frequently exceeds its $T/6$ sub-budget: 12×12 Hard (timeout T=30 s) and 15×15 Medium (T=40 s).

Recovery does not change SR on either benchmark and adds 0.014–0.146 s to mean planning time. For the 30 s setting, the estimated time before Step 2 is below the 11 s feasibility threshold, so the LLM call is skipped. For the 40 s setting, recovery is attempted, but it does not solve any problem that the subsequent relaxation path would otherwise miss. Section V-H analyzes how the feasibility condition protects the relaxation budget.

TABLE 4: Stage 2 ablation: LLM-generated rules with and without LLM failure recovery. SR = success rate; Time = avg. planning time over successful problems (s). Bold = best per row group.

Task	Config	SR	Time (s)
12×12 Hard	Manual	0.880	**3.965**
	LLM-Flax	**0.920**	4.529
	LLM-Flax + Recovery	**0.920**	4.543
15×15 Medium	Manual	**0.967**	**8.740**
	LLM-Flax	**0.967**	9.007
	LLM-Flax + Recovery	**0.967**	9.153

TABLE 5: SR across 8 MazeNamo benchmarks for four LLMs (Stage 1 rules only, same GNN scorer). Avg = simple mean across all 8 tasks. Bold = best per column.

Model	10E	10M	10H	12M	12H	12X	15M	15H	Avg
Flax (manual)	1.000	0.960	0.960	0.960	0.880	0.667	0.967	0.900	0.912
Gemma3-12B	**1.000**	**1.000**	**0.960**	**0.980**	**0.920**	0.700	**0.967**	**1.000**	**0.941**
Qwen2.5-14B	0.940	0.900	0.900	0.920	0.800	**1.000**‡	0.833	**1.000**	0.912
Llama3.1-8B	0.980	**1.000**	0.640	0.960	0.460	0.167‡	0.900	0.933	0.755‡
Mistral-7B	0.820	0.840	0.640	0.820	0.560	0.433‡	0.600	0.567	0.660‡

‡Entries marked with this symbol use earlier 12X measurements and are not directly comparable to the controlled Flax/Gemma3-12B 12X runs.

E. VALIDATION SAFEGUARDS

The model comparison exposed several recurring output errors: string-valued indices, repeated rules, unknown predicate names such as `ismoveempty`, and indices outside a predicate's arity. The validator rejects malformed schemas, non-integer index lists, unknown predicates, and duplicate relaxation rules. Error messages are returned to the model for at most two correction turns, after which generation fails rather than passing an unchecked file to the planner. Gemma3-12B produced the accepted MazeNamo rules on its first attempt. The deterministic deduplication and predicate-filtering passes remain as safeguards for accepted outputs and normalized complementary-rule keys.

F. LLM MODEL COMPARISON (STAGE 1)

To examine model sensitivity, we generate MazeNamo rules with four open-source LLMs served through Ollama. Table 5 reports the resulting SR values with the GNN scorer held fixed.

For the main comparison, we re-ran the 12×12 Expert (12X) Flax and Gemma3-12B configurations under the same code, GNN checkpoint, and test-problem set. This controlled run gives Flax SR 0.667 and Gemma3-12B SR 0.700. Section V-C reports the rule-count ablation using these same controlled conditions. The Qwen2.5-14B, Llama3.1-8B, and Mistral-7B 12X entries come from earlier measurements and are marked ‡. Averages containing these entries are descriptive and are not used for a controlled ranking.

Gemma3-12B has the highest observed mean SR (0.941) and produces the manual relaxation rule on its first attempt.

TABLE 6: Rule-transfer probe with Pure FD, manual rules, and generated rules. Both rule-guided configurations use the same seeded no-guidance scorer. SokomindPlus: n=20, 15×15 split, T=30 s. DifficultLogistics: n=15, T=90 s, measured after the shared relaxation-fallback correction.

Domain	Config	SR	Time (s)
SokomindPlus	Pure FD	**1.000**	15.00
	Manual	0.750	**10.67**
	LLM-Flax	**1.000**	16.96
DifficultLogistics	Pure FD	**1.000**	**41.38**
	Manual	0.667	48.76
	LLM-Flax	0.800	45.83

The Llama3.1-8B configuration contains multiple inappropriate relaxation relations, while the Mistral-7B rules remove heavy as well as light obstacles and alter the intended relaxed state. Across these four models, performance is not monotonic in parameter count; the observed differences instead coincide with differences in generated rule semantics.

G. RULE TRANSFER TO ADDITIONAL DOMAINS

We evaluate Stage 1 on two additional PDDL domains in the Flax codebase. **SokomindPlus** is a Sokoban-style box-pushing domain that shares several relations with MazeNamo, whereas **DifficultLogistics** models trucks, airplanes, packages, locked locations, and stacking. Both domains ship with hand-authored manual rules and 200 training problems, but neither has a domain-specific GNN trained for it. Training such a scorer would require first solving the domain-specific training set; in our SokomindPlus runs, individual optimal-planning calls required 60–300 s. We therefore use the same seeded, untrained uniform-random scorer (`no-guidance`) with both manual and generated rules. This isolates the rule source within each domain, but the resulting values are neither directly comparable with the GNN-guided MazeNamo results nor evidence for scorer transfer.

Before the experiments, we corrected ParsePredicates so that hyphenated names such as `switch-for` and `key-package` are parsed without truncation.

DifficultLogistics also revealed a shared planner-loop error: when Fast Downward proved a relaxed sub-problem unsolvable, both rule configurations terminated instead of continuing to complementary expansion. The corrected loop treats this case as a relaxation that contributes no additional objects and proceeds with the current object set. The same correction is applied to manual and generated rules.

SokomindPlus: Stage 1 transfer Gemma3-12B produces the same relaxation rule as the manual configuration and eight complementary rules instead of one. Generated rules obtain SR 1.000, compared with 0.750 for manual rules; Pure FD also obtains SR 1.000 and has lower mean planning time than the generated-rule configuration.

DifficultLogistics After the shared planner-loop correction, generated rules obtain SR 0.800 and manual rules obtain

SR 0.667. Pure FD remains the strongest configuration, with SR 1.000 and the lowest mean time among successful runs. The result therefore supports transfer of the rule-generation procedure to a structurally different domain, but does not show an advantage over unguided symbolic search in that domain.

A flexibility case study: extending the rule schema itself The DifficultLogistics failure analysis identifies a limitation in the original rule schema. A broad `precond`, such as `obj`, can also match a gating object marked by a narrower predicate such as `key-package`; removing that object can break a global dependency. We added an optional `exclude_precond` field to represent "remove X unless X is also Y." With a domain-independent prompt instruction, Qwen2.5-14B generated:

```
{"precond": {"obj": [0]},
 "exclude_precond": {"key-package": [0]},
 "delete_objects": [0], ...}
```

This rule protects key packages from deletion. Gemma3-12B used the field inconsistently, and other gating dependencies, including switch panels, remain unprotected. We report this extension as a schema case study, not as an additional quantitative result.

H. STAGE 2 BUDGET POLICY ANALYSIS

Runtime recovery competes with the rule-based relaxation step for the same per-problem timeout. The adopted policy therefore checks feasibility before issuing an LLM call:

$$t_{\text{before-step2}} \geq t_{\text{LLM}} + t_{\text{replan,min}} + t_{\text{step2,min}} \tag{1}$$

We set $t_{\text{LLM}} = 5\,\text{s}$, $t_{\text{replan,min}} = 1\,\text{s}$, and $t_{\text{step2,min}} = 5\,\text{s}$, requiring at least 11 s before the Step 2 deadline. The recovery replan is additionally capped at 15% of the total timeout. For $T{=}30\,\text{s}$, only 10 s remain at the decision point, so recovery is skipped. For $T{=}40\,\text{s}$, the check passes and recovery is attempted, but SR remains unchanged. The experiment supports the budget policy as a safeguard for fallback time; it does not show a planning benefit from the recovery proposal itself.

I. EXPLORATORY STAGE 3 PROMPT VARIANTS

We also examined five prompt variants beyond the direct zero-shot scorer. The logged baseline uses $n{=}15$, whereas the exploratory variants use earlier $n{=}50$ runs. We retain these runs to document observed failure mechanisms, not to establish a ranking.

Because the sample sizes and logging setups differ, we draw no pairwise SR conclusions from Table 7. The runs show two recurring constraints: additional reasoning consumes the fixed planning budget, and selective scoring delays objects assigned a low default value.

Goal-biased fact selection Goal-biased selection orders facts about goal objects and positions before other predicates. Relative to alphabetical selection, this changes the mixture of positional and object-property predicates in the 80- or 150-fact input. The unmatched exploratory runs do not isolate whether that change improves score quality.

TABLE 7: Exploratory Stage 3 prompt variants on 12×12 Hard. Direct (alphabetical-80) is the logged baseline ($n{=}15$); rows marked ‡ are earlier measurements ($n{=}50$) and are not directly comparable with the baseline.

Method	SR	Overhead	Observed limitation
Direct, alpha-80 (baseline)	0.533	≈5 s	Incomplete score dictionary
Direct, goal-biased-80	0.660‡	≈7 s	Different predicate coverage
Direct, goal-biased-150	0.620‡	≈10 s	Higher input latency
CoT-full (2-turn, gb-150)	0.100‡	≈12 s	Less time available for planning
CoT-lite (1-turn, gb-150)	0.060‡	≈20 s	Longer response than CoT-full
Top-K ($K{=}40$, gb-150)	0.140‡	≈5 s	Low defaults delay inclusion

Chain-of-Thought (CoT) Two CoT variants were tested. CoT-full uses a 2-turn conversation (Turn 1: free-form path analysis; Turn 2: JSON scores). CoT-lite uses a single turn with an "ANALYSIS:" prefix before the JSON. Both require the model to produce reasoning text in addition to the score dictionary, resulting in approximately 12 s and 20 s of overhead. Under the 30 s timeout, this leaves substantially less time for symbolic planning.

Top-K selective scoring Top-K ($K{=}40$) asks the LLM to score only the K most important objects, defaulting unselected objects to 0.1. This reduces the output JSON from ∼240 to ∼40 entries, cutting generation time to ≈5 s. The selected objects receive scores between 0.6 and 1.0, while approximately 200 unselected objects receive 0.1. Because the threshold starts at 0.81 and decays by $\gamma{=}0.9$, an object scored at 0.1 requires more than 20 decay iterations before inclusion and is unlikely to enter the object set within the Step 1 budget.

Implication The direct scorer does not return a complete score map: 70–99% of objects receive the 0.5 fallback after the single-turn response ends. Additional reasoning increases latency without resolving this incompleteness, whereas Top-K assigns omitted objects a value that is too low for timely threshold decay. The diagnostic therefore motivates bounded batches or pagination rather than a longer prompt alone.

J. BASELINE COMPARISON: PURE FD, PLOI, AND FLAX VARIANTS

This subsection compares four configurations that isolate unguided symbolic search, GNN scoring, and manual or generated rules.

Configurations

- **Pure FD**: Fast Downward with no guidance and no rules. This is the unguided symbolic baseline.
- **PLOI**: GNN-guided `IncrementalPlanner` [3] with no relaxation or complementary rules. Represents the neural baseline without rule-based simplification.
- **Manual (Flax)**: GNN + manually authored rules (1 relaxation, 1 complementary). This is the hand-authored neuro-symbolic baseline.
- **LLM-Flax**: the same GNN + LLM-generated rules (1 relaxation, 7 complementary). This is the Stage 1 configuration reported in Table 1; it is distinct from Full LLM-Flax.

TABLE 8: Comparison of Pure FD, PLOI, manual Flax, and Stage 1 LLM-Flax across eight MazeNamo benchmarks. Time is averaged over successful problems and should be interpreted together with SR. Manual and LLM-Flax values are from Table 1.

Task	Config	SR	Time (s)	Len
10×10 Easy	Pure FD	1.000	0.671	15.4
	PLOI	0.900	0.761	17.6
	Manual	1.000	0.932	17.4
	LLM-Flax	1.000	0.898	17.1
10×10 Medium	Pure FD	1.000	1.400	15.7
	PLOI	0.920	0.840	16.5
	Manual	0.960	1.997	19.8
	LLM-Flax	1.000	0.949	16.6
10×10 Hard	Pure FD	1.000	3.030	19.2
	PLOI	0.820	1.310	19.2
	Manual	0.960	2.200	19.8
	LLM-Flax	0.960	1.600	19.3
12×12 Medium	Pure FD	1.000	6.190	21.1
	PLOI	0.900	1.320	21.8
	Manual	0.960	1.914	22.1
	LLM-Flax	0.980	2.151	22.3
12×12 Hard	Pure FD	1.000	17.59	28.8
	PLOI	0.580	1.270	26.3
	Manual	0.880	3.965	29.8
	LLM-Flax	0.920	4.529	29.2
12×12 Expert	Pure FD	0.333	26.19	29.7
	PLOI	0.433	1.090	26.1
	Manual	0.667	4.528	31.5
	LLM-Flax	0.700	5.982	29.3
15×15 Medium	Pure FD	1.000	27.33	31.8
	PLOI	0.767	5.090	30.0
	Manual	0.967	8.740	30.9
	LLM-Flax	0.967	9.007	31.0
15×15 Hard	Pure FD	0.333	34.37	29.3
	PLOI	0.700	3.750	32.0
	Manual	0.900	11.901	36.2
	LLM-Flax	1.000	10.979	36.0

Main observations Pure FD solves all instances in six of the eight settings, including 12×12 Hard, and must therefore be treated as a competitive baseline rather than a lower bound. Its SR falls to 0.333 on 12×12 Expert and 15×15 Hard, where its mean time over successful runs is 26.2–34.4 s.

PLOI reduces mean time but has lower SR than Pure FD on six settings. Because time is conditioned on success, this difference does not by itself establish greater efficiency: a configuration that solves fewer, easier instances can have a lower mean.

The manual and generated-rule configurations have similar SR throughout the benchmark. Both outperform the rule-free configurations on 12×12 Expert and 15×15 Hard, while Pure FD is strongest on several easier settings. These results support generated rules as a replacement for manual rules within Flax, not as a uniform improvement over Fast Downward.

K. QUALITATIVE VISUALIZATION: PLANNING TRACES

For each benchmark, we show (a) the full problem, (b) zero-shot LLM scores for diagnostic visualization, and (c) the retained object set and plan from the Stage 1 configuration. The scores in column (b) are not used to produce column (c); the planner uses the trained GNN, matching Table 1. The left block covers the four smaller grids (10×10 and 12×12 Medium); the right block covers the four harder grids (12×12 Hard/Expert and 15×15).

Fig. 6 shows one solved instance from each additional domain using generated rules and the no-guidance scorer. SokomindPlus is rendered as a grid; DifficultLogistics is rendered as a city-clustered graph. In the latter trace, the relaxed problem is unsolvable, so the corrected planner retains all 100 objects before computing the final plan.

L. STAGE 3: ZERO-SHOT OBJECT SCORING

Table 9 reports Full LLM-Flax with per-run logging. The Stage 3 sample sizes are smaller than those of the Stage 1 baselines, so the table describes the performance gap but is not a matched statistical comparison. On 12×12 Hard (n=15), Full LLM-Flax obtains SR 0.533 and mean end-to-end time 24.2 s without GNN training data. On 15×15 Hard (n=10), it does not solve an instance within the timeout.

TABLE 9: Full LLM-Flax versus the manual and Stage 1 configurations. Full LLM-Flax uses no manual rules or GNN training data. Its sample sizes are n=15 for 12×12 Hard and n=10 for 15×15 Hard; the corresponding baseline sizes are n=50 and n=30. Time is undefined when no problem is solved.

Task	Config	SR	Time (s)
12×12 Hard	Manual	0.880	**3.965**
	LLM-Flax	**0.920**	4.529
	Full LLM-Flax	0.533	24.21
15×15 Hard	Manual	0.900	11.901
	LLM-Flax	**1.000**	**10.979**
	Full LLM-Flax	0.000	—

The logged prompt contains approximately 3.5K tokens, including the goal, all object names, and up to 80 state facts. Every diagnostic response ends with `finish_reason=stop`, not `length`, but returns scores for only a subset of the listed objects. The resulting mean score coverage is 17% at 12×12 Hard and 9% at 15×15

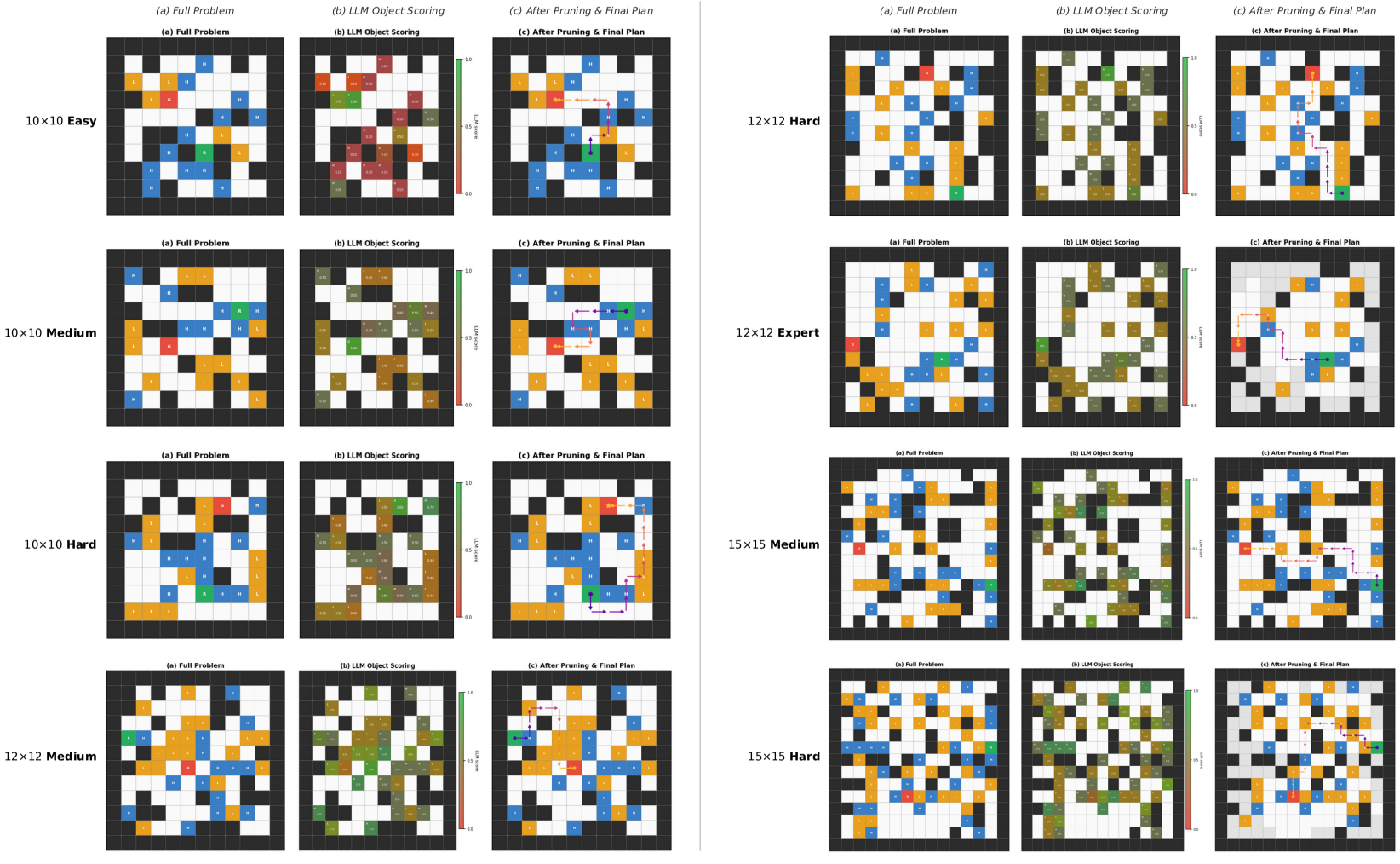

FIGURE 5: Qualitative MazeNamo traces. Each row is one benchmark; columns show **(a)** the full problem, **(b)** zero-shot LLM scores for diagnostic display only, and **(c)** the retained object set and plan produced by the Stage 1 configuration using the trained GNN and generated rules.

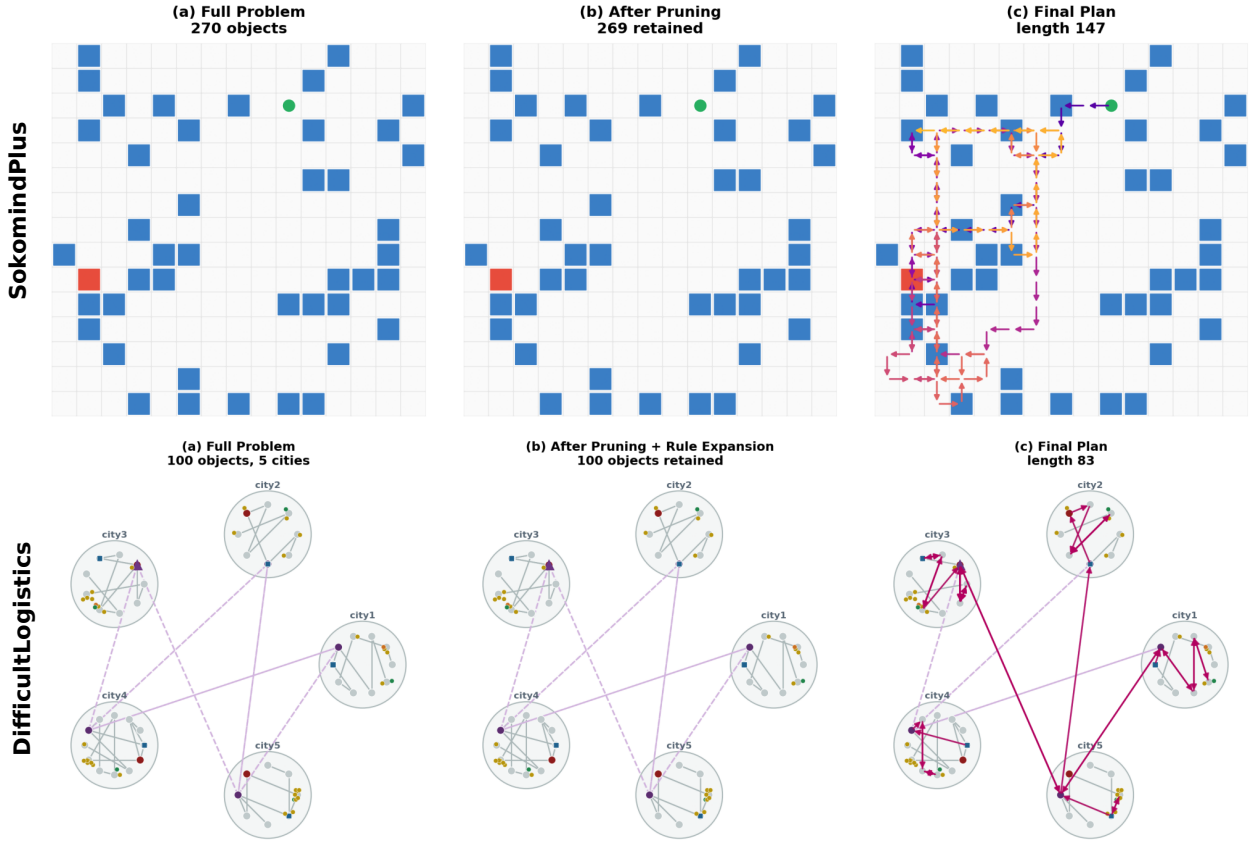

FIGURE 6: Qualitative traces with Gemma3-12B rules and the no-guidance scorer. **Top**: SokomindPlus; blue denotes boxes, red the goal box, and green the robot. **Bottom**: DifficultLogistics; locations are grouped by city, dashed edges denote air links, and column (c) traces the truck and airplane routes. The DifficultLogistics trace retains all 100 objects after the relaxed problem is found unsolvable.

Hard; omitted objects receive the 0.5 fallback. Section V-I reports exploratory prompt variants.

Diagnosing the Stage 3 failure directly We captured the raw score dictionary before applying the fallback on eight problems from each setting (`results/stage3_diagnostic_*.json`). At 12×12, the model scores 2–72 of 233–242 objects; at 15×15, it scores 9–102 of 355–367 objects. Among problems with a known-good manual Flax plan, the returned dictionaries contain 17.9% and 9.6% of the objects used by that plan, on average. This diagnostic points to incomplete single-turn enumeration as one cause of the degradation, while the 80-fact selection may still limit the information available for scoring.

VI. DISCUSSION

Evidence for Stage 1 Stage 1 is the best-supported component of LLM-Flax. Gemma3-12B reproduces the manual MazeNamo relaxation rule and generates complementary relations that yield similar SR across all eight settings. The mean SR difference is +0.029, but the controlled 12×12 Expert ablation reduces the observed gain to one additional solution out of 30 and a 32% time increase. These single-seed results support replacement of manual rules, but not a claim of statistically significant improvement over them. The validator constrains syntax and predicate references; it cannot establish that an accepted rule preserves useful planning semantics. The performance variation across models and the DifficultLogistics gating-object case illustrate this remaining requirement.

Role of the symbolic baseline Pure FD reaches SR 1.000 on six MazeNamo settings and on both additional domains. Its performance falls on the two hardest MazeNamo settings, where successful runs also take 26.2–34.4 s. Manual and generated rules improve SR in those settings, but the result is a benchmark-dependent trade-off rather than a uniform advantage over unguided search. Mean time is conditioned on success and should not be compared independently of SR.

Runtime LLM guidance Stage 2 does not improve SR on either evaluated benchmark. Its contribution is limited to a feasibility policy that prevents the optional call from consuming the relaxation budget. Stage 3 removes GNN training but

performs substantially worse: SR is 0.533 on 12×12 Hard and 0.000 on 15×15 Hard. The diagnostic shows that single-turn JSON generation omits most objects, while alternative prompts either add latency or assign omitted objects scores that delay their inclusion. Batched scoring is a testable next step, but the current results do not support replacing the GNN at the larger setting.

Transfer and adaptation cost The additional-domain experiments compare rule sources under a shared untrained scorer. They show that the rule-generation procedure can produce usable configurations outside MazeNamo, including a structurally different logistics domain, but three domains are insufficient to establish broad domain generalization. Stage 1 also retains the GNN trained on 200 MazeNamo problems. It therefore removes rule-authoring effort, not the separate cost of domain-specific training data. Full LLM-Flax removes both costs only by accepting the Stage 3 performance loss.

Limitations The experiments use one seed, several exploratory tables combine different sample sizes, and the additional-domain study does not include trained domain-specific scorers. A stronger evaluation would use repeated seeds, matched problem sets for every ablation, and more domains with different causal structures. Rule selection on a held-out validation set could also test whether additional complementary relations justify their planning-time cost.

VII. CONCLUSION

We presented three LLM integrations for the Flax neuro-symbolic planner and evaluated their adaptation costs separately. The Stage 1 configuration replaces manually authored rules while retaining the trained GNN. It obtains a descriptive mean SR of 0.941 across eight MazeNamo settings, compared with 0.912 for manual rules, and produces usable rules in two additional-domain probes. The controlled rule-set ablation shows that the measured difference is small and accompanied by additional planning time.

Stage 2 preserves fallback time but does not improve SR. Stage 3 removes domain-specific GNN training data, but its SR falls to 0.533 and 0.000 on the two evaluated settings because the single-turn scorer returns incomplete object dictionaries. Taken together, the experiments support schema-constrained LLM rule generation as a practical reduction in rule-engineering effort. They do not yet support zero-shot LLM scoring as a performance-matched replacement for the trained object scorer.

ACKNOWLEDGMENTS

This research was supported by"Research Base Construction Fund Support Program"funded by Jeonbuk National University in 2026. Also, this work builds directly on the Flax neuro-symbolic planner [5]. The core three-step planning loop, the MazeNamo benchmark environments, and the GNN training infrastructure are all from the original Flax system. We thank the authors for making their code and benchmark publicly available.

REFERENCES

[1] M. Helmert, "The fast downward planning system," vol. 26, 2006, pp. 191–246.
[2] S. Edelkamp and J. Hoffmann, "PDDL2.2: The language for the classical part of the 4th international planning competition," 2004.
[3] T. Silver, K. Allen, A. Lew, L. P. Kaelbling, and J. Tenenbaum, "Planning with learned object importance in large problem instances," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, no. 13, 2021, pp. 11 738–11 746.
[4] Y. Chen et al., "Graph-ploi: Graph neural networks for planning with learned object importance," in International Conference on Robotics and Automation, 2024.
[5] Q. Du, B. Li, Y. Du, S. Su, T. Fu, Z. Zhan, Z. Zhao, and C. Wang, "Fast task planning with neuro-symbolic relaxation," IEEE Robotics and Automation Letters, 2026, arXiv:2507.15975.
[6] T. Brown, B. Mann, N. Ryder, et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.
[7] J. Wei, X. Wang, D. Schuurmans, et al., "Chain-of-thought prompting elicits reasoning in large language models," Advances in Neural Information Processing Systems, vol. 35, pp. 24 824–24 837, 2022.
[8] J. Hoffmann and B. Nebel, "The FF planning system: Fast plan generation through heuristic search," Journal of Artificial Intelligence Research, vol. 14, pp. 253–302, 2001.
[9] R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," Artificial Intelligence, vol. 2, no. 3-4, pp. 189–208, 1971.
[10] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, "PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning," in Proceedings of the International Conference on Automated Planning and Scheduling, vol. 30, 2020, pp. 440–448.
[11] T. Silver, A. Athalye, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Learning neuro-symbolic skills for bilevel planning," in Conference on Robot Learning, 2022, arXiv:2206.10680.
[12] R. Chitnis, T. Silver, J. B. Tenenbaum, T. Lozano-Perez, and L. P. Kaelbling, "Learning neuro-symbolic relational transition models for bilevel planning," in 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2022, pp. 4166–4173.
[13] T. Silver, R. Chitnis, N. Kumar, W. McClinton, T. Lozano-Pérez, L. Kaelbling, and J. B. Tenenbaum, "Predicate invention for bilevel planning," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, no. 10, 2023, pp. 12 120–12 129.
[14] N. Kumar, W. McClinton, R. Chitnis, T. Silver, T. Lozano-Pérez, and L. P. Kaelbling, "Learning efficient abstract planning models that choose what to predict," in Conference on Robot Learning, 2023, pp. 2070–2095.
[15] B. Li, Z. Li, Q. Du, J. Luo, W. Wang, Y. Xie, et al., "LogiCity: Advancing neuro-symbolic AI with abstract urban simulation," in Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 69 840–69 864.
[16] M. Ahn, A. Brohan, N. Brown, et al., "Do as i can, not as i say: Grounding language in robotic affordances," in Conference on Robot Learning, 2022.
[17] B. Liu, Y. Jiang, X. Zhang, et al., "LLM+P: Empowering large language models with optimal planning proficiency," arXiv preprint arXiv:2304.11477, 2023.
[18] M. Zhang et al., "PDDL-based planning with large language models," in International Conference on Automated Planning and Scheduling, 2023.
[19] L. Guan, K. Valmeekam, S. Gao, and S. Kambhampati, "Leveraging pretrained large language models to construct and utilize world models for model-based task planning," in Advances in Neural Information Processing Systems, vol. 36, 2023.
[20] S. Yao, D. Yu, J. Zhao, et al., "Tree of thoughts: Deliberate problem solving with large language models," in Advances in Neural Information Processing Systems, vol. 36, 2023.
[21] S. Cresswell, T. L. McCluskey, and M. M. West, "Acquiring planning domain models using LOCM," The Knowledge Engineering Review, vol. 28, no. 2, pp. 195–213, 2013.
[22] M. Law, A. Russo, and K. Broda, "Inductive learning of answer set programs for autonomous planning," in Proceedings of the 28th International Joint Conference on Artificial Intelligence, 2019.
[23] G. Team, "Gemma 3 technical report," arXiv preprint arXiv:2503.19786, 2025.